

inkmd

Markdown to PDF, pure Python, zero dependencies. MIT-licensed. Deterministic.

```
pip install inkmd
inkmd in.md -o out.pdf
```

That's the whole install. No system packages, no system fonts, no Chrome binary, no apt-get. Works the same on macOS, Linux, Windows, Alpine, AWS Lambda, a locked-down CI runner, or a Steam Deck. Color emoji are bundled, so they render anywhere without a system emoji font.

Quarterly report — Q1 2026

A short paragraph with **bold**, *italic*, ~~struck~~, and `inline code`. This is the sort of mixed-formatting flow that inkmd handles without choking on edge cases or losing kerning between adjacent styled runs.

Revenue is up **18% YoY**. The hard part of the next quarter is shipping the renderer rewrite without slipping the integration date.

Numbers

Region	Q1 2025	Q1 2026	Change
EMEA	£142k	£171k	+20.4%
AMER	£208k	£244k	+17.3%
APAC	£83k	£98k	+18.1%

Notes

- Renderer test coverage now sits at 94% line, 88% branch.
- The migration from the old XML pipeline is paused until Q2 — see the runbook at <https://example.com/runbook> for the rollback plan.

```
def quarterly_growth(prev: float, curr: float) -> float:
    return (curr - prev) / prev
```

Footnote: figures audited 2026-04-12. Contact reports@example.com for the raw spreadsheet.

one page.

See also [examples/inkmd-brief.md](https://github.com/inkmd/inkmd/blob/master/examples/inkmd-brief.md), a two-page project brief written in inkmd-renderable markdown.

What you get

- **A single pure-Python wheel.** No native extensions, no system libraries. Installs in under a second.
- **Faithful CommonMark plus the parts of GFM people actually use:** tables, autolinks, strikethrough, fenced code with language tags. The [supported features](#) section has the full matrix.
- **Color emoji that render anywhere.** 🚀 ✅ 🇯🇵 👍 — single emoji, flags, skin tones, and ZWJ sequences (families, the rainbow flag) all render as color glyphs, inline and in table cells, from a bundled font. No system emoji font required.
- **PDFs that look right.** Real AFM-driven kerning emitted via TJ arrays, clickable links, tinted code-block backgrounds, blockquote rules that stack for nested quotes, table alignment, headings that breathe.
- **Byte-identical output for the same input.** No clocks, no random IDs. Useful for version control, signed PDFs, audit trails, reproducible CI.
- **Two layers of API:** a CLI and a `compile()` / `render_file()` library function. The whole public surface is two functions.

Benchmarks

Measured against WeasyPrint + markdown (the closest pure-Python alternative) on the same input documents, on the same machine. Methodology and full caveats in [BENCHMARKS.md](#); the script is at `scripts/bench.py` and is reproducible.

Metric	inkmd	WeasyPrint	Ratio
Install size (venv)	10.5 MB	74.6 MB	7.1x smaller
Cold-start render, ~1 page	132 ms	814 ms	6.2x faster
Cold-start render, ~11 pages	174 ms	1.40 s	8.0x faster
Peak RSS, ~11 pages	19 MB	122 MB	6.4x lower

WeasyPrint produces slightly smaller PDFs for documents over a few pages (it compresses content streams; inkmd does not). WeasyPrint also supports full Unicode, page-splitting tables, and CSS, which inkmd does not. inkmd does support images as of v0.2 (PNG and JPEG embedding). The right tool depends on your input and your environment — the [comparisons doc](#) has the full picture.

Why this exists

Markdown to PDF is a solved problem in theory and a minefield in practice. Every other tool brings heavy system dependencies that don't survive the trip into an Alpine container, a Lambda function, or a Windows machine without admin rights.

Tool	What goes wrong
wkhtmltopdf	Deprecated since 2023. Unpatched CVEs.
Chrome headless / Puppeteer	200MB+ install. 5 to 15s cold-start latency.
WeasyPrint	Needs Pango, cairo, GObject (350 to 550MB of system packages). Breaks on Alpine and Windows.
Pandoc + LaTeX	3GB texlive install.
PyMuPDF-based tools	Don't build on Alpine musl.
borb	AGPL, so unusable in closed-source or commercial projects without a paid licence.

inkmd runs anywhere Python runs. It's the markdown-to-PDF compiler you'd write yourself with a free weekend if you didn't want to take a dependency on a browser.

For the longer, honest version of how inkmd compares against every realistic alternative (including where inkmd is worse), see [docs/comparisons.md](https://inkmd.com/docs/comparisons.md).

Use cases

- **CI documentation pipelines.** Compile READMEs, release notes, or changelogs to PDF as a build artefact, in a stripped-down container, without apt-get.
- **Agent-generated documents.** LLM agents that need to deliver a PDF (CVs, reports, summaries) can call `inkmd.compile()` directly. No subprocess, no shell-out, no Chrome.
- **Reproducible audit trails.** Hash the markdown, hash the PDF, and the same input gives the same output bytes. Useful for compliance, signed reports, version-controlled docs.
- **Serverless rendering.** Lambda plus zero system dependencies equals a PDF endpoint that cold-starts in well under a second.
- **Restricted environments.** Locked-down CI runners, embedded hardware, anywhere installing a 200MB browser isn't an option.

Status

v0.2, MIT-licensed. 781 tests across 33 files. Stdlib-only, Python 3.9+. Byte-deterministic output.

Conformance against the public spec suites: CommonMark 0.31.2 at 554/652 (85.0%); GFM extensions at 20/28 (71.4%). The full per-section breakdown, the v0.3-tier and v0.4-tier classification of remaining failures, and the real-world-impact framing are in [docs/conformance.md](https://inkmd.com/docs/conformance.md). Threat model in [docs/security.md](https://inkmd.com/docs/security.md). Spec-edge render samples in [docs/gallery/](https://inkmd.com/docs/gallery/), and a real-world rendering gallery (the Ruff README, a Rust Book chapter, a Simon Willison TIL, and inkmd's own README) in [docs/gallery/real-world/](https://inkmd.com/docs/gallery/real-world/).

The v0.2 design principle is **utter consistency**: for any markdown construct the CommonMark spec has a clear answer about, inkmd follows that answer. The conformance percentage is a proxy for "what GitHub showed you is what you get" — it isn't the goal in itself.

Install

From PyPI:

```
pip install inkmd
```

Or grab the single-file zipapp (no pip install required). Each tagged release attaches an `inkmd.pyz` of around 300 KB that you can drop anywhere Python 3.9+ is available:

```
curl -L -o inkmd.pyz
https://github.com/eagrede/inkmd/releases/latest/download/inkmd.pyz
python inkmd.pyz in.md -o out.pdf
```

Or build it yourself from a checkout:

```
python scripts/build_zipapp.py    # produces dist/inkmd.pyz
```

Usage

CLI

```
inkmd in.md -o out.pdf           # file in, file out
inkmd in.md > out.pdf            # file in, stdout out
inkmd < in.md > out.pdf          # stdin in, stdout out
inkmd in.md -o out.pdf --page-size A4 --family times
inkmd in.md -o out.pdf --no-autolinks --no-html
inkmd in.md -o out.pdf --allow-remote-images # fetch http(s) image URLs
inkmd in.md -o out.pdf --allow-unsafe-urls  # disable URL scheme
filter
inkmd in.md -o out.pdf --emoji-fallback drop # only matters in the
no-font build
inkmd --version
```

Library

```
import inkmd

# Compile markdown text to PDF bytes
pdf_bytes = inkmd.compile(md_text)

# Or convert files directly
inkmd.render_file("in.md", "out.pdf")

# Options (same on both functions)
pdf_bytes = inkmd.compile(
    md_text,
    page_size="A4",          # or "letter" (default)
    family="times",          # or "helvetica" (default)
    autolinks=False,        # opt out of GFM bare-URL/email detection
    safe=True,              # URL scheme allow-list (default True)
    html=True,              # inline HTML allow-list (default True)
    allow_remote_images=False, # explicit opt-in to fetch http(s) images
    emoji_fallback="name",   # for emoji the font can't render: "name"
    -> [rocket], or "drop"
)
```

Emoji render as color glyphs out of the box (the font is bundled). Set the `INKMD_NO_EMOJI=1` environment variable to disable emoji rendering; emoji then take the `emoji_fallback` path ("name" ? a [rocket]-style label, or "drop"). The single-file zipapp build ships without the font and behaves the same way.

The public API is intentionally narrow: two functions, no classes to instantiate, no state to manage. The CLI is a thin argparse wrapper around `compile()`.

Supported markdown

CommonMark

Feature	inkmd
Paragraphs with line wrapping	Yes
ATX headings (# to #####)	Yes
Setext headings (=== / ---)	Yes
Ordered lists, arbitrary start	Yes
Unordered lists (- / * / +)	Yes
Nested lists, mixed marker types	Yes

Feature	inkmd
Tight vs. loose list detection	Yes
Blockquotes	Yes
Nested and multi-paragraph blockquotes	Yes
Blockquotes wrapping any block type	Yes
Blockquote lazy continuation	Yes
Fenced code blocks	Yes
Code block language tag (info string)	Yes
Indented code blocks	Yes
Indented code blocks inside list items	Yes
Tabs preserved verbatim inside code	Yes
Code spans (<code>`code`</code> , multi-backtick)	Yes
Emphasis (<i>*</i> , <i>_</i>)	Yes
Strong emphasis (** , __)	Yes
Triple *** becomes nested italic-bold	Yes
Rule of 3 plus intraword-underscore	Yes
Backslash escapes	Yes
Thematic breaks	Yes
Inline links <code>[text](url)</code>	Yes
Inline link titles <code>"..."</code> , <code>'...'</code> , <code>(...)</code>	Yes
Angle-bracket autolinks <code><url></code>	Yes
Reference links <code>[t][ref]</code> , <code>[ref][]</code> , <code>[ref]</code>	Yes
Reference link definitions <code>[ref]: url "title"</code>	Yes
Hard line breaks (two-space or backslash form)	Yes
Soft line breaks	Yes
HTML5 entity references (<code>&am p;</code> , <code>&#42;</code>)	Yes
Images <code>![alt](url)</code>	Yes
Reference-style images <code>![alt][ref]</code>	Yes
Image-inside-link <code>![badge](b.png)](/repo)</code>	Yes
Inline HTML allow-list (<code><sub></code> , <code><mark></code> , <code><u></code> , <code><kbd></code> , <code>
</code>)	Yes

Feature	inkmd
HTML (incl. width + align, and the <p align> figure idiom)	Yes
Block-level raw HTML (<table>, <div> layout, arbitrary tags)	v0.3

GFM extensions

Feature	inkmd
Pipe tables	Yes
Table column alignments	Yes
Bare URL autolinks (https://..., www....)	Yes
Bare host autolinks (host.tld/path)	Yes
Email autolinks (<addr@host>)	Yes
Bare email autolinks (no angle brackets)	v0.3
Bare mailto: / xmpp: schemes	v0.3
Strikethrough ~~text~~ / ~text~	Yes
Task lists - [] / - [x]	Yes
Disallowed-HTML filter	curated subset

Visual output

- Clickable PDF /Link annotations on every URL, inline links and autolinks alike.
- Blue underlined link text.
- Light-grey background tint behind fenced code blocks.
- Thin grey vertical rules for blockquotes. Stacked side-by-side for nested quotes.
- Tinted table headers with full grid borders and per-column alignment.
- AFM-correct kerning emitted via TJ arrays (Helvetica and Times both fully kerned).
- Strikethrough drawn as a thin horizontal bar at glyph mid-height.

Typography

- Helvetica family (default) or Times family. Code uses Courier.
- Standard PDF letter and A4 page sizes.
- WinAnsi character encoding: em-dash, en-dash, curly quotes, ellipsis, most Western European glyphs.
- **Color emoji** render as inline images from a bundled font: single emoji, presentation selectors, regional-indicator flags, skin-tone modifiers, ZWJ sequences (families, the

rainbow flag), and keycaps — inline and in table cells. (Bitmaps scaled to text size; they soften slightly at very large heading sizes.)

- Other codepoints outside WinAnsi (CJK, Cyrillic, Greek, most non-Latin scripts) render as ?. Full text-font embedding would lift this in a later release.

Determinism

inkmd produces **byte-identical** PDF output for the same markdown input on every platform, every Python version, every run. No real-time clocks, no random IDs, no platform-dependent iteration order.

If you hash the markdown and the PDF, the relationship is stable forever. Useful for version-controlled documents, signed/hashed PDFs, reproducible CI builds, and audit trails.

What inkmd doesn't do yet

Feature	When	Why
Text-font embedding for non-Latin scripts (CJK, Cyrillic, ...)	later	v0.2 uses PDF's 14 base fonts for text (WinAnsi). Color emoji are embedded from a bundled font; other non-Latin scripts still render as ?
Block-level raw HTML (<code><table>...</code> <code></table></code> etc.)	v0.3	inkmd v0.2 covers inline HTML via the safe allow-list plus <code></code> (with <code>width/align</code>); arbitrary block-level passthrough is queued
Headers, footers, page numbers	v0.3	Needs a per-page chrome system
Page-splitting for oversized tables	v0.3	Tables place atomically: one taller than a page overflows the bottom, and one with too many columns to fit even at minimum legible width (roughly 25+) overflows the right edge rather than crushing columns into unreadable slivers. Horizontal/vertical table pagination is queued

Feature	When	Why
Blockquote inside a list item	v0.3	> inside an item renders as paragraph text in v0.2; per-item blockquote state coming
RGBA PNG embedding	v0.3	v0.2 supports RGB, grayscale, and indexed PNG (with tRNS transparency); full RGBA alpha is queued
GIF image support	v0.3	LZW decoder + palette resolution
Tagged PDF / PDF/UA accessibility	v1.0+	Under consideration
PDF/A archival format	n/a	Not planned
Math (LaTeX-style)	n/a	Out of scope. Use Pandoc + LaTeX.
Themes / CSS	n/a	Out of scope. Markdown's value is its constraints.

How it works

Four layers, each strictly above the previous:

1. parser is a single-pass container-aware block parser plus a CommonMark inline tokeniser. Produces a frozen-dataclass AST.
2. render lowers AST blocks to RenderedBlock records with runs, spacing, indent, decorations. Carries font and link state through inline nesting.
3. layout wraps runs into pages, positions each PositionedRun against the page coordinate system, emits background rectangles for code blocks, vertical rules for blockquotes, underline plus annotation pairs for links, and bars for strikethrough.
4. pdf serialises pages into PDF bytes. Text via Tj/TJ-with-kerning, graphics via rg/re/f, link annotations via per-page /Annots arrays.

No layer imports a higher one. The whole pipeline is around 9,600 lines of pure-Python logic (including the hand-rolled OpenType reader behind color emoji) plus 4,700 lines of generated AFM kerning tables. That's it. For a deeper walk-through (the emphasis

algorithm, AFM kerning, color-emoji bitmap extraction, determinism mechanics), see [docs/internals.md](https://docs.internals.md). The complexity profile is in LIZARD-AUDIT.md.

A note on font rendering in v0.1 and v0.2

inkmd v0.1 and v0.2 use PDF's **14 base fonts** (Helvetica, Times, Courier, Symbol, ZapfDingbats and their variants). These are spec-mandated to be available in every conforming PDF reader, so we don't ship any font files. The output stays tiny and dependency-free.

The trade-off is that the *actual rendering* depends on which Helvetica (or Times, etc.) the reader's system provides:

- **macOS** ships Helvetica Neue (real Helvetica). Renders as designed.
- **Windows** with Adobe Reader ships real Helvetica. Renders as designed.
- **Linux** typically substitutes Nimbus Sans (URW++'s free Helvetica clone). Renders very similarly but with slightly different side bearings, so spacing between glyphs can look subtly different.
- **Mobile** (iOS / Android) ships system Helvetica or Roboto variants. Mostly fine.

The advance widths are correct everywhere (PDF readers honour the AFM-published metrics), so layout (page breaks, line wrapping, paragraph flow) is identical across systems. What varies is the precise glyph shape *within* each advance-width box, which can produce slightly different visual spacing.

For most use cases this is fine. If you need pixel-identical rendering across every system (signed or archival documents, for example), wait for **v0.3 TTF font embedding**, which will bundle font outlines inside each PDF.

Roadmap

The release tiers are about **what a real user sees**, not about chasing a percentage.

- **v0.1** — Proof of concept: working basic PDFs. **Shipped.**
- **v0.2** — Most sane use cases work; remaining failures are rare and defensible. CommonMark 85%, GFM extensions 71%. Adds reference links, images (PNG + JPEG + indexed PNG with transparency), **color emoji** (single, flags, skin tones, ZWJ sequences, keycaps — inline and in tables), task lists, inline HTML allow-list, hard line breaks, indented code blocks (including inside list items), URL scheme filter, tab preservation, image-inside-link.
- **v0.3** — Visually identical for the user even where spec tests still fail. Adds block-level raw HTML pass-through, blockquote-inside-list, headers/footers/page numbers, page-splitting for oversized tables, text-font embedding for non-Latin scripts (CJK, Cyrillic, ...), full RGBA PNG, GIF.
- **v0.4** — 100% CommonMark and 100% GFM extensions. The long-tail spec-corner cases.
- **v1.0 and beyond** — Tagged PDF, accessibility, TOC generation, cross-references.

PDF/A and similar under consideration.

Licence

MIT. See [LICENSE](#).

The bundled color-emoji font is **Noto Color Emoji** (© Google), distributed under the [SIL Open Font License 1.1](#) — a separate, permissive licence from inkmd's own MIT code. It is shipped unmodified.

Acknowledgements

The 14 standard PDF fonts and their AFM metric files are public-domain artefacts published by Adobe ([adobe-type-tools/Core14_AFM](#)s). Color emoji are rendered from Google's [Noto Color Emoji](#) (SIL OFL 1.1). PDF format reference: ISO 32000-1.

About

Built by [Dylan Moir](#) — architecture, problem decomposition, and implementation directed end-to-end through AI tooling (Claude Code), with every output reviewed. Sister projects: [Nightjar](#), an autonomous LLM agent with a defence-in-depth security architecture, and [TORCH](#), an AI-orchestrated IDE.

If inkmd saves you a fight with WeasyPrint or a 200MB Chrome install in your CI, a star on the repo is plenty.